LOOQBOX DATA CHALLENGE

Technical Solution Report

Candidate	Matheus Amorim
Role / Process	Analista de Dados e BI (Python/SQL - Remoto)
Repository / PR	https://github.com/looqbox/data-challenge/pull/8
Submission date	23/05/2026
Main tools	Python, SQL, pandas, matplotlib, SQLAlchemy

1. Executive Summary

This document outlines my solution to the Looqbox Data Challenge by means of SQL and Python. The work is divided into SQL questions and three case studies (practical examples) where dynamic data retrieval, provided queries for data manipulation, and a custom visualisation will all be explored via the IMDB_movies table. All SQL questions were explored via DBeaver connecting to the MySQL challenge database. The case studies were developed via Python using the pandas library for data manipulation and matplotlib for data visualisation. The purpose of this report is to provide the queries, code, results, and rationale for how I solved each component of the challenge.

2. Environment and Tools

The solution was developed using Python and SQL with a direct connection to the challenge MySQL database. The main Python libraries used were pandas, SQLAlchemy, PyMySQL, and matplotlib.

Component	Description
Database	MySQL - challenge schema
Python libraries	pandas, SQLAlchemy, matplotlib
Output files	SQL results, tables, and charts included in this report
Repository	https://github.com/looqbox/data-challenge/pull/8

3. SQL Questions

3.1 Top 10 Most Expensive Products

Question: What are the 10 most expensive products in the company?

```
SELECT dp.PRODUCT_NAME, dp.PRODUCT_VAL, dp.DEP_NAME
FROM data_product dp
ORDER BY dp.PRODUCT_VAL DESC
LIMIT 10
```

	PRODUCT_NAME	PRODUCT_VAL	DEP_NAME
1	Whisky Escoces THE MACALLAN Ruby Garrafa 700ml com Caixa	741.99	BEBIDAS
2	Whisky Escoces JOHNNIE WALKER Blue Label Garrafa 750ml	735.9	BEBIDAS
3	Cafeteira Expresso 3 CORACOES Tres Modo Vermelho	499	BEBIDAS
4	Vinho Portugues Tinto Vintage QUINTA DO CRASTO Garrafa 750ml	445.9	BEBIDAS
5	Escova Dental Eletrica ORAL B D34 Professional Care 5000 110v	399.9	PERFUMARIA
6	Champagne Rose VEUVE CLICQUOT PONSARDIM Garrafa 750ml	366.9	MERCEARIA
7	Champagne Frances Brut Imperial MOET Rose Garrafa 750ml	359.9	MERCEARIA
8	Conjunto de Panelas Allegra em Inox TRAMONTINA 5 Pecas Gratis Utensilios 5 Pecas	359	MERCEARIA
9	Whisky Escoces CHIVAS REGAL 18 Anos Garrafa 750ml	329.9	BEBIDAS
10	Champagne Frances Brut Imperial MOET & CHANDON Garrafa 750ml	315.9	MERCEARIA

3.2 Sections from BEBIDAS and PADARIA Departments

Question: What sections do the 'BEBIDAS' and 'PADARIA' departments have?

```
SELECT DISTINCT dp.section_name, dp.dep_name, dp.section_cod
FROM data_product dp
WHERE dp.dep_name IN ('BEBIDAS', 'PADARIA')
```

	section_name	dep_name	section_cod
1	BEBIDAS	BEBIDAS	4
2	VINHOS	BEBIDAS	30
3	DOCES-E-SOBREMESAS	PADARIA	8
4	QUEIJOS-E-FRIOS	PADARIA	22
5	CERVEJAS	BEBIDAS	29
6	PADARIA	PADARIA	19
7	REFRESCOS	BEBIDAS	31
8	GESTANTE	PADARIA	27

3.3 Total Product Sales by Business Area in Q1 2019

Question: What was the total sale of products (in $) of each Business Area in the first quarter of 2019?

```
SELECT dsc.business_name, dsc.business_code, SUM(dps.sales_value)
FROM data_store_cad dsc
INNER JOIN data_product_sales dps
        ON dsc.store_code = dps.store_code
WHERE dps.date BETWEEN '2019-01-01' AND '2019-03-31'
GROUP BY dsc.business_name, dsc.BUSINESS_CODE
```

	business_name	business_code	SUM(dps.sales_value)
1	Varejo	1	81,032,347.65
2	Farma	4	81,776,691.73
3	Atacado	5	80,384,884.6
4	Posto	3	32,072,326.4
5	Proximidade	2	80,171,122.8

4. Case 1 - Dynamic Data Retrieval Function

The objective of this case was to create a flexible Python function capable of retrieving data from data_product_sales based on optional filters for product code, store code, and date range.

```
import pandas as pd
from sqlalchemy import create_engine

engine = create_engine(
    "mysql+pymysql://looqbox-challenge:looq-challenge@35.199.115.174:3306/looqbox-challenge"
)

def retrieve_data(product_code = None, store_code = None, date = None):

    aux = []
    where_clause = ''

    if product_code:
        aux.append(f'dps.product_code = {product_code}')

    if store_code:
        aux.append(f'dps.store_code = {store_code}')

    if date and len(date) == 2:
        aux.append(f"dps.date BETWEEN '{date[0]}' AND '{date[1]}'")

    if aux:
        where_clause = 'WHERE ' + ' AND '.join(aux)

    query = f"""
    SELECT *
    FROM data_product_sales dps
    {where_clause}
    """

    df = pd.read_sql(query, engine)

    return df

result = retrieve_data()
```

```
print(result)
```

Example with nothing in the function:

```
❯ python case_first_question.py
         STORE_CODE  PRODUCT_CODE        DATE  SALES_VALUE  SALES_QTY
0                 1            18  2019-01-01       708.50       65.0
1                 1            18  2019-01-02      1297.10      119.0
2                 1            18  2019-01-03      1144.50      105.0
3                 1            18  2019-01-04      1090.00      100.0
4                 1            18  2019-01-05       893.80       82.0
...             ...           ...         ...          ...        ...
2173128           9        241404  2019-12-27      7671.75      193.0
2173129           9        241404  2019-12-28      6201.00      156.0
2173130           9        241404  2019-12-29      4889.25      123.0
2173131           9        241404  2019-12-30      4730.25      119.0
2173132           9        241404  2019-12-31      5127.75      129.0

[2173133 rows x 5 columns]
```

Example with "result = retrieve_data(18, 1, ['2019-01-01', '2019-01-15'])":

```
❯ python case_first_question.py
    STORE_CODE  PRODUCT_CODE        DATE  SALES_VALUE  SALES_QTY
0            1            18  2019-01-01        708.5       65.0
1            1            18  2019-01-02       1297.1      119.0
2            1            18  2019-01-03       1144.5      105.0
3            1            18  2019-01-04       1090.0      100.0
4            1            18  2019-01-05        893.8       82.0
5            1            18  2019-01-06        741.2       68.0
6            1            18  2019-01-07        654.0       60.0
7            1            18  2019-01-08        741.2       68.0
8            1            18  2019-01-09       1373.4      126.0
9            1            18  2019-01-10       1068.2       98.0
10           1            18  2019-01-11       1057.3       97.0
11           1            18  2019-01-12        806.6       74.0
12           1            18  2019-01-13        686.7       63.0
13           1            18  2019-01-14        697.6       64.0
14           1            18  2019-01-15        763.0       70.0
```

5. Case 2 - Ready Queries and Average Ticket

The client provided two ready-to-go queries. The queries were used as the data source, and the requested date range was applied in Python. The final result contains store name, business category, and TM.

In this solution, TM was interpreted as average ticket and calculated as:

```
TM = SUM(SALES_VALUE) / SUM(SALES_QTY)
```

```
import pandas as pd
from sqlalchemy import create_engine

engine = create_engine(
    "mysql+pymysql://looqbox-challenge:looq-challenge@35.199.115.174:3306/looqbox-challenge"
)

query1 = f"""
SELECT
      STORE_CODE,
      STORE_NAME,
      START_DATE,
      END_DATE,
      BUSINESS_NAME,
      BUSINESS_CODE
FROM data_store_cad;
"""

query2 = f"""
SELECT
       STORE_CODE,
       DATE,
       SALES_VALUE,
       SALES_QTY
FROM data_store_sales
WHERE DATE BETWEEN '2019-01-01' AND '2019-12-31'
"""

df1 = pd.read_sql(query1, engine)
df2 = pd.read_sql(query2, engine)

df1.columns = [col.lower() for col in df1.columns]
df2.columns = [col.lower() for col in df2.columns]

df2['date'] = pd.to_datetime(df2['date'])

df2 = df2[
    (df2['date'] >= '2019-10-01') &
    (df2['date'] <= '2019-12-31')
]

df_temp = df2.groupby('store_code')[['sales_value', 'sales_qty']].sum().reset_index()

df_temp['TM'] = df_temp['sales_value'] / df_temp['sales_qty']

df_temp['TM'] = df_temp['TM'].round(2)

df_final = pd.merge(
    df1[['store_code', 'store_name', 'business_name']],
    df_temp[['store_code', 'TM']],
    on = 'store_code',
    how = 'left'
)
```

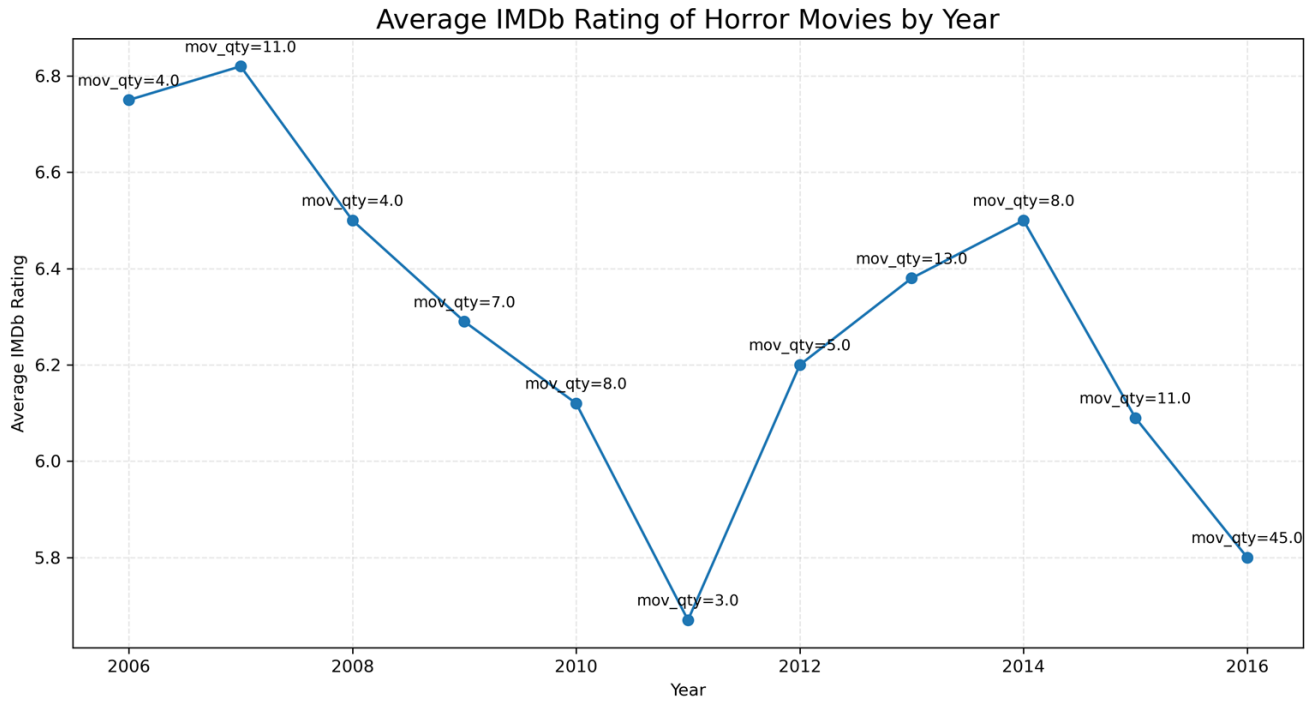
```
df_final = df_final[['store_name', 'business_name', 'TM']]

df_final = df_final.rename(columns={
    'store_name': 'Loja',
    'business_name': 'Categoria',
})

df_final = df_final.sort_values('Loja')

print(df_final.to_string(index=False))
```

```
❯ python case_second_question.py
          Loja   Categoria    TM
         Bahia     Atacado 15.39
       Bangkok       Posto 13.67
         Belem Proximidade 15.37
        Berlin Proximidade 15.39
  Buenos Aires     Atacado 15.39
       Chicago      Varejo 15.53
         Dubai     Atacado 15.39
     Hong Kong       Farma 26.35
        London       Farma 28.99
         Madri       Farma 29.03
         Miami       Posto 13.67
      New York Proximidade 15.39
         Paris Proximidade 15.39
 Rio de Janeiro       Farma 29.59
          Roma      Varejo 15.39
      Salvador     Atacado 15.39
     Sao Paulo      Varejo 15.39
        Sidney       Posto 13.67
         Tokio      Varejo 15.39
     Vancouver       Posto 13.67
```

6. Case 3 - Custom Visualization: Horror Movies by Year

For the custom visualization, the selected analysis investigates whether older horror movies tend to have higher IMDb ratings than newer ones. The chart groups horror movies by release year and compares the average rating over time.

```
import pandas as pd
import matplotlib.pyplot as plt
from sqlalchemy import create_engine

engine = create_engine(
    "mysql+pymysql://looqbox-challenge:looq-challenge@35.199.115.174:3306/looqbox-challenge"
)

query = """
SELECT
    Title,
    Genre,
    Year,
    Rating
FROM IMDB_movies
WHERE Genre LIKE '%%Horror%%'
  AND Rating IS NOT NULL
  AND Year IS NOT NULL;
"""

df = pd.read_sql(query, engine)

df_year = (
    df
    .groupby("Year", as_index=False)
    .agg(
        avg_rating=("Rating", "mean"),
        movie_count=("Title", "count")
    )
)

df_year["avg_rating"] = df_year["avg_rating"].round(2)

plt.figure(figsize=(11, 6))

plt.plot(df_year["Year"], df_year["avg_rating"], marker="o")

for _, row in df_year.iterrows():
    plt.text(
        row["Year"],
        row["avg_rating"] + 0.03,
        f'mov_qty={row["movie_count"]}',
        ha='center',
        fontsize=9
    )

plt.title("Average IMDb Rating of Horror Movies by Year", fontsize=16)
plt.xlabel("Year")
plt.ylabel("Average IMDb Rating")

plt.grid(True, linestyle="--", alpha=0.3)
plt.tight_layout()
plt.savefig("horror_rating_by_year.png", dpi=300, bbox_inches="tight")
plt.show()

print(df_year)
```



Explanation of visualization choice:

My choice of chart was due to my personal interest in the horror genre. The main idea was to verify whether older horror movies have higher average ratings than newer ones. To do this, I filtered only the movies that have `"Horror"` in their genre and grouped the data by release year. Then, I calculated the average IMDb Rating for each year. I also included the number of movies from each year in the chart, because years with fewer movies can generate less reliable averages. Looking at the chart, it is possible to observe that the older years, especially 2006 and 2007, had higher average ratings. However, in the following years there is a decrease and then some variation in the ratings, with some more recent years also having reasonable averages, such as 2013 and 2014. Therefore, the chart indicates that the older horror movies in the dataset have higher average ratings on average, but this conclusion should be analyzed carefully, since the number of movies varies considerably from year to year.

7. Notes About AI / LLM Assistance

The use of AI tools was limited to punctual support during the development process.

I used AI tools only as occasional support during the challenge. In Case 2, I used AI to look up the meaning of "TM" and confirm that it referred to the average ticket calculation. In Case 3, I used AI to adjust minor details in the visualization, such as chart formatting and the best way to present the number of movies per year, but the insight of comparing older movies with newer ones to check for a possible trend was my own idea.

I also used AI to support the organization of this document, mainly for the template structure and presentation suggestions. The data exploration, SQL queries, case interpretation, implementation logic, and final decisions were done and reviewed by me.

8. Final Checklist

- SQL Question 1 answered with query and result
- SQL Question 2 answered with query and result
- SQL Question 3 answered with query and result
- Case 1 code included and tested
- Case 2 code included and final table shown
- Case 3 chart included and explained
- AI assistance disclosure included
- Repository / Pull Request link included